

MecaniQA(São Paulo) – Modelos de Aprendizado de Máquina

Autor: Denilson Santos da Silva
Gabriel Arlisson de Souza Santos Torres
João Arthur Nascimento Mascarenhas
Tony Cleriston Oliveira dos Santos Junior
Vinicius Cerqueira Oliveira

Agenda

Apresentação da sequência dos principais tópicos da apresentação. Abrange desde o contexto da MecâniQA até os modelos, validação, métricas e atributos temporais.

1. Contexto da MecâniQA

Cenário e desafios da plataforma.

2. Baselines e Data Leakage

Modelo Naive, média móvel e prevenção de informações futuras.

3. Validação e métricas

Validação temporal, MAE, RMSE e MAPE.

4. Engenharia de atributos temporais

Lags, Rolling Window e tratamento de valores ausentes.

5. Considerações finais

Resultados e próximos passos.

MecâniQA – Automotive Tech



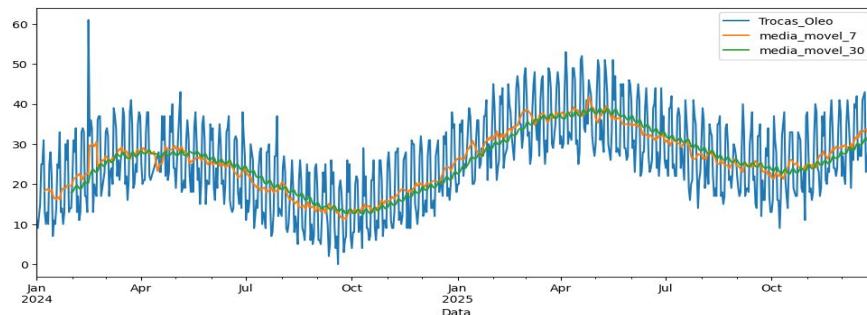
- Problema: Picos inesperados de demanda nas oficinas e auto centers.
- Impactos: Falta de peças no estoque e ociosidade dos mecânicos.
- Etapa anterior: Compreensão dos dados e criação de baselines simples.
- Desafio atual: Avaliar rigorosamente os baselines desenvolvidos na OAT 1.
- Objetivo: Medir o desempenho dos modelos e identificar o quanto as previsões estão próximas da realidade.



Baselines e prevenção de Data Leakage



- Objetivo: Criar os primeiros modelos de comparação (Baselines).
- Modelo Naive: Previsão de hoje = valor observado ontem.
- Média Móvel: Utiliza os valores anteriores, como uma janela de 7 dias com `rolling()`.
- Data Leakage: Evitar que o modelo utilize informações do futuro na previsão por meio do método `.shift(1)`: Usa apenas dados anteriores para realizar a previsão.



Para o modelo enxergar o passado, criamos atributos defasados "lags" e uma média móvel para usar como entrada

```
1 df['lag_1'] = df['Trocas_Oleo'].shift(1)
   df['lag_7'] = df['Trocas_Oleo'].shift(7)
   df['lag_30'] = df['Trocas_Oleo'].shift(30)
```

Criando a janela rolante sem vaziar dados futuros

```
1 df['rolling_mean_7'] = df['Trocas_Oleo'].shift(1).rolling(7).mean()
   df['rolling_mean_30'] = df['Trocas_Oleo'].shift(1).rolling(30).mean()
```

o `TimeSeriesSplit` mede a estabilidade do erro ao longo do tempo.

```
1 df['previsao_naive'] = df['lag_1']
   df['previsao_naive_sazonal_7'] = df['lag_7']
   df['previsao_naive_sazonal_30'] = df['lag_30']
   df['media_movel_7'] = df['rolling_mean_7']
   df['media_movel_30'] = df['rolling_mean_30']
```

Validação Temporal e Métricas

- Time Series Split: Mantém a ordem cronológica, treinando com o passado e testando o futuro.
- MAE: Mostra o erro médio na mesma unidade do negócio, sendo fácil de interpretar.
- RMSE: Dá maior peso aos erros grandes e evidencia valores fora da curva.
- MAPE: Apresenta o erro médio em porcentagem.
- Aplicação: Avaliar os modelos Naive e Média Móvel e gerar o boletim de desempenho.

Criando função avaliadora

```
def avaliar_modelo(y_true, y_pred, nome_modelo):  
    mae = mean_absolute_error(y_true, y_pred)  
    mse = mean_squared_error(y_true, y_pred)  
    rmse = np.sqrt(mse)  
    mape = mean_absolute_percentage_error(y_true, y_pred) * 100  
  
    print(f"Resultados do {nome_modelo} - MAE: {mae:.2f}, RMSE: {rmse:.2f}, MAPE: {mape:.2f}%")
```

```
print("\nDesempenho Geral dos Baselines:")  
avaliar_modelo(y_real, y_pred_naive, "Baseline Naive ")  
avaliar_modelo(y_real, y_pred_mm7, "Baseline Média Móvel 7 Dias")  
  
...  
Desempenho Geral dos Baselines:  
Resultados do Baseline Naive - MAE: 6.67, RMSE: 9.11, MAPE: 4497175091525490.50%  
Resultados do Baseline Média Móvel 7 Dias - MAE: 7.38, RMSE: 8.21, MAPE: 8627233849048878.00%
```

Feature Engineering



- Objetivo: Preparar os dados para os modelos de Machine Learning.
- Lags: Criar atributos com valores passados, como Lag 1, Lag 7 e Lag 30.
- Rolling Window: Criar médias móveis como novas features, evitando o uso de dados futuros.
- Tratamento de dados: Remover ou tratar os valores NaN gerados pelos Lags.
- Enriquecimento: Adicionar atributos temporais ao dataset para melhorar as previsões.

Para o modelo enxergar o passado, criamos atributos defasados "lags" e uma média móvel para usar como entrada

```
df['lag_1'] = df['Trocas_Oleo'].shift(1)
df['lag_7'] = df['Trocas_Oleo'].shift(7)
df['lag_30'] = df['Trocas_Oleo'].shift(30)
```

Criando a janela rolante sem vaziar dados futuros

```
df['rolling_mean_7'] = df['Trocas_Oleo'].shift(1).rolling(7).mean()
df['rolling_mean_30'] = df['Trocas_Oleo'].shift(1).rolling(30).mean()
```

Tirando os valores nulos

```
df_limpo = df.dropna().copy()
```

```
print("\nDesempenho Geral dos Baselines:")
avaliar_modelo(y_real, y_pred_naive, "Naive (Lag 1)")
avaliar_modelo(y_real, y_pred_naive_7, "Naive Sazonal Semanal (Lag 7)")
avaliar_modelo(y_real, y_pred_naive_30, "Naive Sazonal Mensal (Lag 30)")
avaliar_modelo(y_real, y_pred_mm7, "Média Móvel 7 Dias")
avaliar_modelo(y_real, y_pred_mm30, "Média Móvel 30 Dias")
```

```
Desempenho Geral dos Baselines:
Resultados do Naive (Lag 1) - MAE: 6.67, RMSE: 9.11, MAPE: 4497175091525490.50%
Resultados do Naive Sazonal Semanal (Lag 7) - MAE: 3.67, RMSE: 5.07, MAPE: 3212267922518203.00%
Resultados do Naive Sazonal Mensal (Lag 30) - MAE: 10.54, RMSE: 12.74, MAPE: 5139628676029146.00%
Resultados do Média Móvel 7 Dias - MAE: 7.38, RMSE: 8.21, MAPE: 8627233849048878.00%
Resultados do Média Móvel 30 Dias - MAE: 7.33, RMSE: 8.26, MAPE: 8630293151832227.00%
```

Feature Engineering



df_limpo.head(15)

Trocas_Oleo Manutencao_Motor lag_1 lag_7 lag_30 rolling_mean_7 rolling_mean_30 previsao_naive previsao_naive_sazonal_7 previsao_naive_sazonal_30 media_mov_7 media_mov_30

Data

2024-01-31	14.0	5.0	14.0	10.0	11.0	19.571429	17.900000	14.0	10.0	11.0	19.571429	17.900000
2024-02-01	21.0	13.0	14.0	13.0	9.0	20.142857	18.000000	14.0	13.0	9.0	20.142857	18.000000
2024-02-02	31.0	12.0	21.0	30.0	12.0	21.285714	18.400000	21.0	30.0	12.0	21.285714	18.400000
2024-02-03	28.0	13.0	31.0	26.0	15.0	21.428571	19.033333	31.0	26.0	15.0	21.428571	19.033333
2024-02-04	34.0	12.0	28.0	31.0	25.0	21.714286	19.466667	28.0	31.0	25.0	21.714286	19.466667
2024-02-05	13.0	5.0	34.0	13.0	25.0	22.142857	19.766667	34.0	13.0	25.0	22.142857	19.766667
2024-02-06	17.0	10.0	13.0	14.0	31.0	22.142857	19.366667	13.0	14.0	31.0	22.142857	19.366667
2024-02-07	11.0	3.0	17.0	14.0	13.0	22.571429	18.900000	17.0	14.0	13.0	22.571429	18.900000
2024-02-08	13.0	8.0	11.0	21.0	10.0	22.142857	18.833333	11.0	21.0	10.0	22.142857	18.833333
2024-02-09	33.0	14.0	13.0	31.0	13.0	21.000000	18.933333	13.0	31.0	13.0	21.000000	18.933333
2024-02-10	34.0	12.0	33.0	28.0	10.0	21.285714	19.600000	33.0	28.0	10.0	21.285714	19.600000
2024-02-11	33.0	14.0	34.0	34.0	25.0	22.142857	20.400000	34.0	34.0	25.0	22.142857	20.400000
2024-02-12	17.0	8.0	33.0	13.0	28.0	22.000000	20.666667	33.0	13.0	28.0	22.000000	20.666667
2024-02-13	17.0	8.0	17.0	17.0	22.0	22.571429	20.300000	17.0	17.0	22.0	22.571429	20.300000
2024-02-14	13.0	2.0	17.0	11.0	7.0	22.571429	20.133333	17.0	11.0	7.0	22.571429	20.133333

Próximas etapas:

[Gerar código com df_limpo](#)

[New interactive sheet](#)

Considerações finais



- Foram estruturados os primeiros modelos de referência: Naive e Média Móvel.
- A abordagem adotada preserva a ordem temporal dos dados e evita Data Leakage por meio do uso de informações exclusivamente anteriores à previsão.
- Foi definida uma estratégia de validação temporal, utilizando o passado para treinamento e o futuro para teste.
- MAE, RMSE e MAPE foram estabelecidas como métricas para avaliar o desempenho das previsões.
- A base foi enriquecida com lags e janelas móveis, criando atributos temporais para futuras etapas de modelagem.
- Com isso, foi estabelecida uma base inicial de comparação e avaliação para a evolução dos modelos preditivos da MecâniQA.

Referências

- HYNDMAN, Rob J.; ATHANASOPOULOS, George. **Forecasting: Principles and Practice**. 3. ed. Melbourne: OTexts, 2021. Disponível em: <https://otexts.com/fpp3/>. Acesso em: 23 set. 2026.
- PANDAS DEVELOPMENT TEAM. **pandas documentation**. Disponível em: <https://pandas.pydata.org/docs/>. Acesso em: 23 set. 2026.
- PANDAS DEVELOPMENT TEAM. **pandas.DataFrame.shift**. Disponível em: <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.shift.html>. Acesso em: 23 set. 2026.
- PANDAS DEVELOPMENT TEAM. **pandas.Series.rolling**. Disponível em: <https://pandas.pydata.org/docs/reference/api/pandas.Series.rolling.html>. Acesso em: 23 set. 2026.
- SCIKIT-LEARN DEVELOPERS. **TimeSeriesSplit**. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html. Acesso em: 23 set. 2026.
- SCIKIT-LEARN DEVELOPERS. **Model evaluation: quantifying the quality of predictions**. Disponível em: https://scikit-learn.org/stable/modules/model_evaluation.html. Acesso em: 23 set. 2026.